

Weiterentwicklung eines selbstfahrenden Fahrzeugs mit Lidar und anderen Sensoren

STUDIENARBEIT

des Studienganges Informatik
an der Dualen Hochschule Baden-Württemberg Ravensburg Campus Friedrichshafen

von

Elena Schwarzbach, 3830156, TIK23

Sonia Sinaci, 8366601, TIK23

Scott Jonathan Hebach, 8958697, TIT23

Marius Maurer, 9339665, TIT23

11. Juli 2026

Bearbeitungszeitraum: x Wochen

Gutachter der Dualen Hochschule: Titel Claudia Zinser

Eidesstattliche Erklärung

gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden-Württemberg vom 29.09.2017.

Ich versichere hiermit, dass ich meine Bachelorarbeit (bzw. Projektarbeit oder Studienarbeit bzw. Hausarbeit) mit dem Thema:

Weiterentwicklung eines selbstfahrenden Fahrzeugs mit Lidar und anderen Sensoren

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Friedrichshafen, 11. Juli 2026

Elena Schwarzbach

Sonia Sinaci

Scott Jonathan Hebach

Marius Maurer

Genderhinweis

In der vorliegenden Arbeit wird aus Gründen der sprachlichen Ökonomie und zur Sicherung eines einheitlichen Schriftbildes auf eine durchgängige geschlechts-spezifische Differenzierung verzichtet. Sämtliche Personen-bezeichnungen im generischen Maskulinum verstehen sich als geschlechtsneutral und schließen alle Geschlechter gleichermaßen ein. Diese Entscheidung basiert rein auf redaktionellen Überlegungen und spiegelt keine Geringschätzung gegenüber weiblichen oder nicht-binären Personen wider. Die Autoren sind sich der Wirkung von Sprache auf die Wahrnehmung bewusst und verfolgen mit dieser Wahl das Ziel, die Komplexität des Textes zugunsten der inhaltlichen Klarheit zu reduzieren.

Inhaltsverzeichnis

Abkürzungsverzeichnis	I
Abbildungsverzeichnis	I
Tabellenverzeichnis	II
1 Einleitung	1
1.1 Motivation	1
1.2 Zielsetzung	1
1.3 Problemstellung	2
2 Ist-Zustand	3
2.1 App-Steuerung (Elena)	3
2.2 Funktionierend	3
2.3 Probleme	3
3 Technikkorridor	4
3.1 Hardwarevergleich Raspberry Pi 4 und Raspberry Pi 5	4
3.2 Vergleich unterschiedlicher Betriebssysteme für Raspberry Pi	6
3.3 Softwarearchitektur	7
3.4 Betrachtete Systemarchitekturen	9
3.5 Kommunikationskonzept	10
3.6 Entscheidung und Begründung	11
3.7 Fazit	12
4 Autoumbau	14
4.1 Umverteilung des Gewichtes	14
4.1.1 Hochkante Einlage in den Mittelteil	14
4.1.2 Frankenstein Umbau	15
4.1.3 Einkaufsrads Lösung	16
4.1.4 Nutzwertanalyse zur Gewichtsumverteilung	16
4.2 Gegenständen ausweichen	17
4.2.1 Mathematische Formeln für die Berechnung der Fahrbahn	17
4.2.2 Berechnung Lenkwinkel	20

4.2.3	Entwicklung der Hinderniserkennung	21
4.2.4	Implementierung der Gauß-basierten Hindernisvermeidung . . .	22

Abbildungsverzeichnis

1	Parabel Funktion	18
2	Exponential Funktion	19
3	Gaussche Glocke Funktion	19

Tabellenverzeichnis

1	Direkter Hardwarevergleich zwischen Raspberry Pi 4 und Raspberry Pi 5	5
2	Vergleich relevanter Betriebssystemoptionen	7
3	Vergleich zwischen nativer ROS2-Installation und Docker-basierter ROS2-Architektur	9
4	Bewertung betrachteter Systemarchitekturen	10
5	Nutzwertanalyse zur Gewichtsverteilung	17

1 Einleitung

In diesem Kapitel werden die Motivation, die Zielsetzung sowie die Ausgangssituation des Projekts beschrieben. Darüber hinaus werden die bestehenden Herausforderungen dargestellt, die im Rahmen der Weiterentwicklung des Fahrzeugs gelöst werden sollen.

1.1 Motivation

Autonome Systeme gewinnen in vielen Bereichen zunehmend an Bedeutung. Neben dem Einsatz im Straßenverkehr werden sie auch in der Logistik, der Industrie sowie in verschiedenen Assistenzsystemen verwendet. Sie können Aufgaben selbstständig ausführen, ihre Umgebung erfassen und auf Veränderungen reagieren. Dadurch lassen sich Prozesse effizienter und zuverlässiger gestalten.

Das vorliegende Projekt bietet die Möglichkeit, zentrale Aspekte autonomer Systeme praxisnah zu untersuchen und umzusetzen. Dabei werden sowohl die technische Weiterentwicklung eines bestehenden Fahrzeugs als auch die Integration autonomer Fahrfunktionen betrachtet. Aufgrund der hohen Praxisrelevanz und des Zukunftspotenzials autonomer Systeme wurde dieses Projekt ausgewählt.

1.2 Zielsetzung

Studierende der Elektrotechnik und Informatik haben bereits ein Fahrzeug entwickelt, das als Grundlage für dieses Projekt dient. Das Fahrzeug verfügt über verschiedene Sensoren zur Umfelderkennung, darunter einen LiDAR-Sensor sowie mehrere Ultraschallsensoren.

Ziel des Projekts ist die Weiterentwicklung des bestehenden Systems. Das Fahrzeug soll zukünftig sowohl manuell über ein Smartphone gesteuert als auch autonom betrieben werden können. Hierfür wird ein Selbstfahrmodus implementiert, der eine eigenständige Navigation innerhalb einer definierten Umgebung ermöglicht. Dabei soll das Fahrzeug Hindernisse zuverlässig erkennen und geeignete Ausweichmanöver ausführen.

Die vorhandene Sensorik bildet die Grundlage für die Umsetzung dieser Funktionen.

Durch die Erweiterung der Hard- und Software soll ein robuster Betrieb in beiden Betriebsarten gewährleistet werden.

1.3 Problemstellung

Vor Beginn der eigentlichen Weiterentwicklung mussten zunächst verschiedene Probleme in den Bereichen Elektronik, Mechanik und Software analysiert werden.

Im Bereich der Elektronik existierte kein vollständiger Verdrahtungsplan, wodurch die Nachvollziehbarkeit der Schaltung eingeschränkt war. Zusätzlich wurden beschädigte Kabel sowie Leitungen gefunden, deren Zweck nicht eindeutig erkennbar war. Darüber hinaus stellte die begrenzte Akkulaufzeit eine Einschränkung für einen längeren Fahrzeugbetrieb dar.

Auch die mechanische Konstruktion wies mehrere Schwachstellen auf. Durch das hohe Gewicht des vorderen Aufbaus war das Fahrzeug um etwa 7° nach vorne geneigt. Zusätzlich erschwerte das kompakte Design Wartungs- und Reparaturarbeiten. Einige Sensorhalterungen waren beschädigt oder fehlten vollständig, wodurch die Sensorik nicht zuverlässig montiert werden konnte.

Im Softwarebereich führten die verwendeten Anwendungen zu einer hohen Speicher- und Arbeitsspeicherauslastung. Darüber hinaus war die vorhandene Dokumentation unvollständig, was die Einarbeitung in das bestehende System sowie dessen Erweiterung erschwerte.

Die genannten Probleme mussten zunächst analysiert und behoben werden, um eine zuverlässige Grundlage für die Umsetzung der neuen Funktionen zu schaffen.

2 Ist-Zustand

2.1 App-Steuerung (Elena)

Die App besitzt bereits eine theoretische Steuerung über die Bewegungssensoren des Smartphones. Dafür wird in `gyroscope_data.dart` die Bibliothek `motion_sensors` verwendet. Das Widget `getGyroscope` liest bereits die Werte für Pitch, Roll und Yaw aus. Sobald neue Sensordaten eintreffen, werden Pitch und Roll auf den Bereich von -1.0 bis 1.0 begrenzt. Anschließend werden diese Werte direkt als Steuerbefehle über den `WebSocketManager` an das Fahrzeug gesendet. Dabei soll der Roll-Wert die Geschwindigkeit steuern und der Pitch-Wert für den Lenkwinkel verwendet werden. Der Yaw-Wert wird für die Steuerung nicht verwendet. Zusätzlich zeigt die App die aktuellen Sensorwerte für Pitch, Roll und Yaw auf der Oberfläche an.

In diesem Stand ist die Funktion zwar im Code angelegt, funktioniert praktisch aber noch nicht. Ein zentrales Problem liegt darin, dass die App von `motion_sensors.orientation` abhängig ist. Das Paket `motion_sensors` ist jedoch veraltet und liefert keine Werte. Außerdem werden die Sensorwerte direkt weitergegeben, ohne sie vorher zu filtern oder zu glätten. Dadurch könnten bereits kleinste Bewegungen oder Messungenauigkeiten die Steuerung beeinflussen.

Weitere Probleme sind, dass es keine Deadzone für unbeabsichtigte kleine Bewegungen gibt und keine Möglichkeit vorgesehen ist, die Empfindlichkeit der Steuerung anzupassen. Auch eine einfache Korrektur falsch herum reagierender Achsen ist nicht vorbereitet.

2.2 Funktionierend

2.3 Probleme

3 Technikkorridor

Der Technikkorridor beschreibt den technisch sinnvollen Lösungsraum für die Realisierung eines autonomen Fahrzeugs auf Raspberry-Pi-Basis. Ziel ist es, aus den verfügbaren Hardware-, Betriebssystem- und Softwareoptionen eine Plattform abzuleiten, die für echtzeitnahe Sensorverarbeitung, Kamerabildverarbeitung, Batteriebetrieb, Wartbarkeit und Erweiterbarkeit geeignet ist.

Im Mittelpunkt steht dabei die Frage, welche Kombination aus Einplatinencomputer, Betriebssystem, Robotik-Middleware und optionaler Containerisierung für ein Studien- oder Forschungsprojekt den besten Kompromiss aus Leistungsfähigkeit, Ressourcenbedarf und Entwicklungsaufwand bietet. Für autonome Fahrzeuge sind insbesondere parallele Sensordatenströme, Bildverarbeitung, stabile Kommunikation zwischen Softwarekomponenten sowie eine robuste und wartbare Systemstruktur relevant.

Der Raspberry Pi 4 und der Raspberry Pi 5 stellen dabei zwei naheliegende Hardwareoptionen dar. Während der Raspberry Pi 4 weiterhin für bestehende oder kostenkritische Anwendungen geeignet ist, bietet der Raspberry Pi 5 durch seine höhere CPU-Leistung, bessere I/O-Anbindung und Erweiterbarkeit über PCIe eine deutlich leistungsfähigere Grundlage für Robotik- und autonome Fahrzeugsysteme [zotero-item-143] [zotero-item-144]. Ergänzend beeinflussen Betriebssystemwahl, Softwarearchitektur und Kommunikationskonzept maßgeblich die Praxistauglichkeit des Gesamtsystems.

3.1 Hardwarevergleich Raspberry Pi 4 und Raspberry Pi 5

Der Hardwarevergleich dient dazu, die Eignung des Raspberry Pi 4 und des Raspberry Pi 5 für ein autonomes Fahrzeug einzuordnen. Beide Plattformen besitzen eine ähnliche Grundidee als kompakte Einplatinencomputer, unterscheiden sich jedoch deutlich hinsichtlich Rechenleistung, Speicheranbindung, I/O-Architektur und Erweiterbarkeit.

Der Raspberry Pi 4 basiert auf einer Cortex-A72-Plattform und ist mit bis zu 8 GB LPDDR4-Arbeitsspeicher verfügbar. Er bietet zwei USB-3.0- und zwei USB-2.0-Anschlüsse, Gigabit-Ethernet sowie zwei Micro-HDMI-Ausgänge für Dual-Display-Betrieb bis 4K. Damit ist er für allgemeine IoT-, Embedded- und Server-Anwendungen weiterhin brauchbar. Für rechenintensive Aufgaben wie parallele Sensorverarbeitung, Bildauswertung oder mehrere gleichzeitig laufende Robotik-Komponenten stößt er jedoch

früher an Leistungsgrenzen [zotero-item-143].

Der Raspberry Pi 5 stellt demgegenüber die technisch modernere Plattform dar. Er verwendet eine Cortex-A76-basierte CPU und erreicht nach Herstellerangaben eine etwa zwei- bis dreifache CPU-Leistungssteigerung gegenüber dem Raspberry Pi 4 [zotero-item-144]. Hinzu kommen LPDDR4X-Arbeitsspeicher, der neue RP1-I/O-Controller, eine PCIe-2.0-x1-Schnittstelle, eine integrierte Echtzeituhr sowie ein Power-Button auf dem Board [zotero-item-144]. Besonders relevant ist die PCIe-Schnittstelle, da sie über entsprechende Adapter den Einsatz von NVMe-SSDs ermöglicht. Dadurch können Logdaten, Kartenmaterial, Container-Images oder Build-Artefakte deutlich performanter gespeichert und gelesen werden.

Auch externe Benchmarks bestätigen den deutlichen Leistungssprung. In Geekbench-6-Messungen erreicht der Raspberry Pi 5 im Single-Core-Test etwa 764 bis 774 Punkte, während der Raspberry Pi 4 bei ungefähr 340 Punkten liegt. Dies entspricht etwa dem 2,4-fachen Leistungsniveau [zotero-item-146]. Zusätzlich werden dem Raspberry Pi 5 im Vergleich zum Raspberry Pi 4 eine höhere Speicher- und SD-Karten-Performance sowie eine verbesserte Kamera- und I/O-Anbindung zugeschrieben [zotero-item-147].

Tabelle 1: Direkter Hardwarevergleich zwischen Raspberry Pi 4 und Raspberry Pi 5

Merkmal	Raspberry Pi 4	Raspberry Pi 5
CPU	Cortex-A72	Cortex-A76
Arbeitsspeicher	LPDDR4, bis zu 8 GB	LPDDR4X
PCIe	nicht vorhanden	PCIe 2.0 x1
I/O-Architektur	klassisch integriert	RP1-I/O-Controller
USB-Bandbreite	geringer	höher
Kameraanbindung	geringer	verbessert
RTC	nicht hervorgehoben	auf dem Board vorhanden
Power-Button	nicht hervorgehoben	auf dem Board vorhanden

Für ein autonomes Fahrzeug ist der Raspberry Pi 5 vor allem aufgrund seiner höheren CPU-Reserven und verbesserten I/O-Fähigkeiten geeigneter. Sensorfusion, Kamerabildverarbeitung und mehrere parallel laufende Softwarekomponenten profitieren stark von dieser zusätzlichen Leistung. Der Raspberry Pi 4 bleibt hingegen eine sinnvolle Option für bestehende Projekte oder kostenkritische Anwendungen, wenn die Anforderungen an Bildverarbeitung, Datenrate und Erweiterbarkeit begrenzt bleiben.

Bei der Energieversorgung ist zwischen Netzteilanforderung und tatsächlicher Leistungsaufnahme zu unterscheiden. Für den Raspberry Pi 5 nennt der Hersteller eine

Stromversorgung über USB-C mit 5 V und 5 A über USB Power Delivery [zotero-item-144]. Für den Raspberry Pi 4 Model B werden 5 V und 3 A über USB-C oder GPIO angegeben; ein gutes 2,5-A-Netzteil kann genügen, wenn die USB-Peripherie weniger als 500 mA benötigt [zotero-item-150]. Diese Angaben sind jedoch Versorgungsanforderungen und dürfen nicht unmittelbar als durchschnittlicher Energieverbrauch interpretiert werden.

3.2 Vergleich unterschiedlicher Betriebssysteme für Raspberry Pi

Die Wahl des Betriebssystems beeinflusst Ressourcenverbrauch, Hardwareintegration, ROS2-Kompatibilität, Wartbarkeit und Entwicklungsaufwand. Für ein autonomes Fahrzeug sind insbesondere schlanke Systeme mit guter Treiberunterstützung und stabiler Langzeitnutzung relevant. Betrachtet werden Raspberry Pi OS Lite, Ubuntu Server, Ubuntu Desktop und DietPi. Zusätzlich wird die Bedeutung von ARM64 für ROS2 berücksichtigt.

Raspberry Pi OS Lite ist das offiziell naheliegende Betriebssystem für viele Raspberry-Pi-Anwendungsfälle. Es verzichtet auf eine Desktop-Umgebung und eignet sich daher besonders für Headless-Server, Embedded-Anwendungen und batteriebetriebene Systeme. Der geringe Ressourcenbedarf reduziert RAM-Verbrauch, Hintergrundlast und tendenziell auch den Energiebedarf. Raspberry Pi OS Lite 64 Bit wird für Raspberry Pi 3, 4 und 5 empfohlen und bietet eine sehr gute Hardwareintegration [zotero-item-149] [zotero-item-151].

Ubuntu Server ist vor allem dann attraktiv, wenn das Projekt stark am Ubuntu- und ROS-Ökosystem ausgerichtet ist. Viele ROS2-Installationsanleitungen, Pakete und Entwicklungsabläufe beziehen sich direkt auf Ubuntu. Dadurch kann Ubuntu Server den Entwicklungsaufwand reduzieren, wenn bereits Linux-Server-Know-how vorhanden ist oder wenn professionelle DevOps-nahe Workflows genutzt werden sollen [zotero-item-148]. Ubuntu nennt als Systemanforderungen für Server-Installationen unter anderem einen Mindestbedarf an Arbeitsspeicher und Massenspeicher, wobei für produktive Robotikprojekte ausreichend schneller Speicher eingeplant werden sollte [zotero-item-153].

Ubuntu Desktop eignet sich besonders für Entwicklung, Simulation und Debugging direkt auf dem Raspberry Pi. Werkzeuge wie RViz und Gazebo können lokal genutzt werden, was bei der Entwicklung und Analyse von ROS2-Systemen hilfreich ist. Gleich-

zeitig benötigt Ubuntu Desktop deutlich mehr RAM, Speicherplatz und Rechenleistung als ein Headless-System. Für den produktiven Fahrzeugeinsatz ist es daher weniger geeignet, kann aber als Entwicklungsplattform sinnvoll sein [zotero-item-154].

DietPi ist eine besonders schlanke Alternative für ressourcenschonende Embedded- oder Server-Setups. Es kann interessant sein, wenn minimale Systemlast wichtiger ist als maximale offizielle Raspberry-Pi-Hardwareintegration. Da in der übergebenen BibTeX-Datei jedoch kein eigener DietPi-Eintrag enthalten ist, sollte für eine endgültige wissenschaftliche Fassung noch ein passender BibTeX-Eintrag ergänzt werden.

Für ROS2 ist ARM64 besonders relevant, da diese Architektur im Raspberry-Pi-Kontext eine wichtige Rolle spielt. Eine direkte native ROS2-Installation ist insbesondere auf Ubuntu naheliegend. Raspberry Pi OS 64 Bit kann dagegen in Kombination mit Docker genutzt werden, um Ubuntu-basierte ROS2-Container auszuführen [zotero-item-156].

Tabelle 2: Vergleich relevanter Betriebssystemoptionen

Kriterium	Pi OS Lite	Ubuntu Server	Ubuntu Desktop	DietPi
RAM-Bedarf	sehr gering	mittel	hoch	sehr gering
Strombedarf	gering	mittel	hoch	gering
ROS2-Nähe	gut	sehr gut	sehr gut	abhängig vom Se
Hardwareintegration	sehr gut	gut	gut	gut bis mittel
Entwicklung am Gerät	eingeschränkt	eingeschränkt	sehr gut	eingeschränkt
SSD/NVMe-Einsatz	optional sinnvoll	sinnvoll	sehr sinnvoll	optional

Für ein autonomes Fahrzeug mit Batteriebetrieb und begrenzten Ressourcen ist Raspberry Pi OS Lite 64 Bit besonders geeignet. Es bietet eine gute Kombination aus geringer Systemlast und sehr guter Hardwareunterstützung. Ubuntu Server bleibt eine starke Alternative, wenn ROS2-Kompatibilität, vorhandenes Ubuntu-Know-how oder spätere Skalierung im Vordergrund stehen. Ubuntu Desktop sollte primär als Entwicklungs- und Debugging-Umgebung betrachtet werden.

3.3 Softwarearchitektur

Die Softwarearchitektur legt fest, wie Sensorik, Aktorik, Wahrnehmung, Steuerung und Diagnose logisch strukturiert und miteinander verbunden werden. Für autonome Fahrzeuge bietet sich eine modulare Architektur an, bei der einzelne Funktionen in getrennten Softwarekomponenten ausgeführt werden. Dadurch können Sensorverarbeitung,

Lokalisierung, Fahrentscheidung und Motorsteuerung unabhängig entwickelt, getestet und gewartet werden.

ROS2 eignet sich als zentrale Middleware für ein solches System. Es unterstützt die modulare Kommunikation zwischen Knoten und ist in Robotik, autonomen Fahrzeugsystemen und Forschungsprojekten weit verbreitet. Sensoren, Aktoren, Steuerungslogik und Diagnosefunktionen können in ROS2 als getrennte Nodes realisiert werden. Die Kommunikation erfolgt über Topics, Services und Actions, wodurch eine klare Trennung zwischen Datenströmen, Steuerbefehlen und länger laufenden Aktionen möglich wird [**zotero-item-156**].

Docker kann ergänzend eingesetzt werden, um Softwareabhängigkeiten und Laufzeitumgebungen in Container zu kapseln. Dies verbessert Reproduzierbarkeit, Wartbarkeit und Deployment, insbesondere wenn mehrere Entwickler beteiligt sind oder mehrere Softwarekomponenten unabhängig voneinander aktualisiert werden müssen. Für kleinere Einzelentwicklerprojekte kann Docker jedoch zusätzliche Komplexität verursachen.

Wissenschaftliche Untersuchungen zeigen, dass Linux-Container bei CPU- und Speicheroperationen häufig nahezu native Performance erreichen können, während relevanter Overhead vor allem bei I/O- und Netzwerkpfaden entstehen kann [**instituteofelectricalandele**]. Spezifische Untersuchungen auf verschiedenen Raspberry-Pi-Modellen zeigen zudem, dass Docker grundsätzlich auch auf ressourcenbegrenzten Raspberry-Pi-Systemen einsetzbar ist. Die praktische Skalierbarkeit hängt jedoch stark vom verfügbaren Arbeitsspeicher und von der jeweiligen Container-Workload ab [**gameSSPerformanceEvaluationDock**].

Der Ressourcenverbrauch von Containern sollte projektbezogen gemessen werden. Docker stellt hierfür unter anderem Metriken zu CPU-Auslastung, Speicherverbrauch, Netzwerk-I/O und Block-I/O bereit [**zotero-item-163**]. Außerdem verursacht Docker keinen festen, pauschalen RAM-Overhead pro Container. Der tatsächliche Ressourcenbedarf hängt vom Container-Image, von der laufenden Anwendung, von der Anzahl paralleler Container und von gesetzten Ressourcenlimits ab [**dockerinc.RuntimeMetrics**].

Für ein Studienprojekt oder ein Einzelentwicklerprojekt überwiegen häufig die Vorteile einer nativen ROS2-Installation: geringere Komplexität, direkter Hardwarezugriff und weniger Ressourcenbedarf. Docker lohnt sich insbesondere dann, wenn reproduzierbare Deployments, klare Trennung mehrerer Komponenten, Teamarbeit oder CI/CD-Prozesse relevant sind.

Tabelle 3: Vergleich zwischen nativer ROS2-Installation und Docker-basierter ROS2-Architektur

Kriterium	ROS2 nativ	Docker + ROS2
Energieeffizienz	besser	tendenziell schlechter
RAM-Verbrauch	geringer	höher bzw. workloadequabhängig
Deployment	aufwendiger	einfacher reproduzierbar
Wartbarkeit	geringer	höher
Teamarbeit	weniger komfortabel	besser geeignet
Reproduzierbarkeit	geringer	höher
Hardwarezugriff	direkter	konfigurationsabhängig
Komplexität	geringer	höher

3.4 Betrachtete Systemarchitekturen

Aus der Kombination von Hardware, Betriebssystem, ROS2 und optionaler Containerisierung ergeben sich mehrere technisch sinnvolle Systemarchitekturen. Diese unterscheiden sich insbesondere hinsichtlich Ressourcenbedarf, Wartbarkeit, Entwicklungsaufwand und Eignung für den produktiven Fahrzeugeinsatz.

Die erste Variante ist Raspberry Pi OS Lite 64 Bit mit nativer ROS2-Installation. Diese Architektur ist besonders ressourcenschonend und hardwarenah. Sie eignet sich für Einzelentwicklungen, Studienprojekte und batteriebetriebene Fahrzeuge, bei denen geringe Systemlast und einfache Wartung wichtiger sind als vollständig containerisierte Deployments. Der direkte Hardwarezugriff kann insbesondere bei GPIO, Kamera, Sensorik und Aktorik vorteilhaft sein.

Die zweite Variante ist Raspberry Pi OS Lite 64 Bit mit Docker und ROS2. Sie kombiniert die gute Raspberry-Pi-Hardwareintegration mit reproduzierbaren Softwareumgebungen. Diese Architektur ist sinnvoll, wenn mehrere ROS2-Komponenten sauber voneinander getrennt werden sollen oder wenn ein konsistentes Deployment auf mehreren Geräten erforderlich ist. Der Nachteil liegt im höheren Administrationsaufwand und im zusätzlichen Ressourcenbedarf.

Die dritte Variante ist Ubuntu Server mit nativer ROS2-Installation. Sie ist besonders geeignet, wenn das Projekt stark am Ubuntu- und ROS2-Ökosystem orientiert ist. Viele ROS2-Anleitungen und Pakete sind für Ubuntu ausgelegt, wodurch die Einrichtung erleichtert werden kann [zotero-item-156]. Gegenüber Raspberry Pi OS Lite ist jedoch mit einer höheren Systemlast zu rechnen.

Die vierte Variante ist Ubuntu Server mit Docker und ROS2. Diese Architektur bietet

hohe Modularität und Reproduzierbarkeit. Sie ist besonders für Teamprojekte, CI/CD-Prozesse und verteilte Softwarekomponenten geeignet. Gleichzeitig ist sie die komplexeste der betrachteten produktiven Varianten und benötigt sorgfältige Ressourcenüberwachung, insbesondere auf einem Raspberry Pi [gamePerformanceEvaluationDocker2023] [zotero-item-163].

Die fünfte Variante ist Ubuntu Desktop mit ROS2. Sie eignet sich vor allem als Entwicklungsplattform für grafische Werkzeuge wie RViz oder Gazebo. Für den produktiven Fahrzeugeinsatz ist sie wegen des höheren RAM-, Speicher- und Energiebedarfs weniger attraktiv [zotero-item-154].

Tabelle 4: Bewertung betrachteter Systemarchitekturen

Architektur	Stärken	Einschränkungen
Pi OS Lite + ROS2	effizient, hardwarenah	weniger reproduzierbares Deployment
Pi OS Lite + Docker + ROS2	wartbar, reproduzierbar	mehr Komplexität und Ressourcenbedarf
Ubuntu Server + ROS2	gute ROS2-Nähe	höhere Systemlast
Ubuntu Server + Docker + ROS2	modular, teamfähig	komplexeste Variante
Ubuntu Desktop + ROS2	gut für Entwicklung	ungeeignet für ressourcenarmen Betrieb

3.5 Kommunikationskonzept

Das Kommunikationskonzept beschreibt, wie auf das Fahrzeug zugegriffen wird und wie Steuerung, Diagnose und Wartung erfolgen. Für ein autonomes Fahrzeug ist eine robuste und möglichst unabhängige Kommunikationslösung wichtig, da das System auch außerhalb vorhandener Netzwerkinfrastruktur zuverlässig erreichbar bleiben soll.

Ein lokaler WLAN-Access-Point auf dem Raspberry Pi stellt hierfür eine geeignete Lösung dar. Das Fahrzeug baut dabei ein eigenes WLAN auf, mit dem sich ein Entwicklungsrechner, Tablet oder Smartphone verbinden kann. Dadurch sind Steuerung, Parametrierung, Diagnose und Logzugriff auch dann möglich, wenn kein externes Netzwerk vorhanden ist. Diese Betriebsart ist besonders praktisch für Tests in Laborumgebungen, auf Teststrecken oder in Bereichen ohne zuverlässige WLAN-Infrastruktur.

Alternativ kann der Raspberry Pi als WLAN-Client in ein vorhandenes Netzwerk eingebunden werden. Dies ist sinnvoll, wenn das Fahrzeug dauerhaft in einer Labor- oder Campusinfrastruktur betrieben wird. Der Vorteil liegt in der einfachen Integration in bestehende Netzwerke. Der Nachteil besteht darin, dass die Erreichbarkeit von der vorhandenen Infrastruktur abhängt.

Ethernet bietet eine robuste und latenzarme Verbindung, ist jedoch kabelgebunden. Es eignet sich daher besonders für Laborbetrieb, stationäre Tests, Einrichtung und Debugging, aber weniger für den mobilen Fahrzeugeinsatz. LTE oder 5G ermöglichen entfernte Einsätze und Fernzugriff, erhöhen jedoch Kosten, Energiebedarf und Systemkomplexität. Für ROS2-Systeme ist außerdem zu beachten, dass die zugrunde liegende DDS-Kommunikation netzwerkabhängig konfiguriert werden muss, insbesondere wenn mehrere Systeme über bestehende Infrastruktur miteinander kommunizieren.

Für das betrachtete autonome Fahrzeug ist daher ein lokaler WLAN-Access-Point als primäres Kommunikationskonzept sinnvoll. Ergänzend kann Ethernet für Einrichtung und Debugging vorgesehen werden. WLAN-Client-Betrieb oder Mobilfunk können optional ergänzt werden, wenn der konkrete Anwendungsfall dies erfordert.

3.6 Entscheidung und Begründung

Auf Basis des Hardwarevergleichs, der Betriebssystembewertung und der betrachteten Softwarearchitekturen wird folgende Zielplattform empfohlen:

- Raspberry Pi 5 als Hardwareplattform,
- Raspberry Pi OS Lite 64 Bit als Betriebssystem,
- ROS2 als zentrale Robotik-Middleware,
- Docker optional, abhängig von Teamgröße, Deployment-Anforderungen und Projektkomplexität,
- lokaler WLAN-Access-Point als primäres Kommunikationskonzept.

Der Raspberry Pi 5 wird gegenüber dem Raspberry Pi 4 bevorzugt, da er deutlich höhere CPU-Reserven, bessere I/O-Leistung, eine modernere Speicheranbindung und PCIe-Erweiterbarkeit bietet [zotero-item-144]. Gerade bei Sensorfusion, Kamerabildverarbeitung und parallelen ROS2-Knoten sind diese Reserven entscheidend. Die gemessenen Benchmark-Unterschiede zeigen zudem, dass der Raspberry Pi 5 im Single-Core-Bereich deutlich leistungsfähiger ist als der Raspberry Pi 4 [zotero-item-146].

Raspberry Pi OS Lite 64 Bit wird gewählt, weil es eine geringe Systemlast mit sehr guter Hardwareintegration verbindet. Der Verzicht auf eine Desktop-Umgebung reduziert RAM-Verbrauch, Hintergrundprozesse und Energiebedarf. Dies ist für ein batteriebetriebenes autonomes Fahrzeug besonders wichtig [zotero-item-149].

ROS2 wird als Softwarebasis gewählt, weil es eine modulare Architektur für Robotiksysteme ermöglicht und die Kommunikation zwischen Sensorik, Aktorik, Wahrnehmung und Steuerung strukturiert unterstützt [zotero-item-156]. Die native Installation ist für ein Einzelentwickler- oder Studienprojekt zunächst die effizienteste Lösung, da sie weniger Systemkomplexität verursacht und direkten Hardwarezugriff erleichtert.

Docker wird nicht als zwingender Bestandteil der Zielarchitektur festgelegt, sondern als optionale Ergänzung bewertet. Der Einsatz ist sinnvoll, wenn mehrere Entwickler beteiligt sind, mehrere ROS2-Komponenten sauber getrennt werden sollen oder reproduzierbare Deployments erforderlich sind. Wenn hingegen maximale Akkulaufzeit, maximale Performance oder ein möglichst einfaches System im Vordergrund stehen, ist eine native ROS2-Installation vorzuziehen. Diese differenzierte Entscheidung ist wichtig, da Docker zwar Wartbarkeit und Reproduzierbarkeit verbessert, aber zusätzlichen Administrationsaufwand und workloadequabhängigen Ressourcenbedarf verursacht [gameessPerformanceEvaluationDocker2023] [dockerinc.RuntimeMetrics].

3.7 Fazit

Der Technikkorridor zeigt, dass der Raspberry Pi 5 mit Raspberry Pi OS Lite 64 Bit und ROS2 den besten technologischen Ausgangspunkt für ein autonomes Fahrzeug im Studien- oder Forschungsumfeld darstellt. Diese Kombination verbindet hohe Rechen- und I/O-Reserven mit geringer Systemlast, guter Hardwareunterstützung und hoher Zukunftssicherheit.

Der Raspberry Pi 4 bleibt für bestehende oder kostenkritische Projekte weiterhin nutzbar, ist für leistungsorientierte autonome Systeme jedoch weniger geeignet. Der Raspberry Pi 5 bietet durch seine stärkere CPU, den RP1-I/O-Controller, bessere Kamera- und USB-Anbindung sowie PCIe/NVMe-Erweiterbarkeit die deutlich tragfähigere Plattform [zotero-item-144] [zotero-item-147].

Bei der Betriebssystemwahl ist Raspberry Pi OS Lite 64 Bit für den produktiven Fahrzeugeinsatz besonders geeignet, da es ressourcenschonend ist und eine sehr gu-

te Raspberry-Pi-Hardwareintegration bietet [zotero-item-149]. Ubuntu Server bleibt eine gute Alternative, wenn die Nähe zum ROS2- und Ubuntu-Ökosystem wichtiger ist als minimale Systemlast. Ubuntu Desktop ist vor allem als Entwicklungsplattform für Visualisierung, Simulation und Debugging sinnvoll.

Docker verbessert Wartbarkeit, Reproduzierbarkeit und Skalierbarkeit, sollte aber nicht pauschal als Pflichtbestandteil der Architektur betrachtet werden. Für Einzelentwicklungen und batteriebetriebene Studienprojekte ist Raspberry Pi OS Lite mit nativer ROS2-Installation meist die effizienteste Lösung. Docker lohnt sich insbesondere bei Teamprojekten, CI/CD-Prozessen oder mehreren verteilten Softwarekomponenten. Der empfohlene Technikkorridor lautet daher: Raspberry Pi 5, Raspberry Pi OS Lite 64 Bit, ROS2 nativ als Basis und Docker als optionale Erweiterung.

4 Autoumbau

4.1 Umverteilung des Gewichtes

Da der Lidar Sensor eine Neigung um 7° nach vorne hat (schwerer, ungestützter Vorbau), muss das Gewicht neu verteilt werden. Hierfür werden drei Möglichkeiten herausgearbeitet und in Tabelle 5 verglichen.

4.1.1 Hochkante Einlage in den Mittelteil

Art der Änderung

- Der Vorderbau wird mechanisch begradigt: die Bereiche um die Schraubpunkte werden abgefeilt, bis eine rechteckige, sauber einpassbare Form entsteht.
- Der Vorderbau wird anschließend hochkant in den Mittelteil integriert.
- Fixierung über einen Winkel hinter der Servolenkung, damit die Einheit verwindungssteif sitzt und Kräfte definiert eingeleitet werden.
- Das Batteriefach wird hochkant ausgerichtet und in Richtung Motor verschoben, um den Bauraum besser zu nutzen und den Schwerpunkt sinnvoll zu verlagern.
- Leitungsführung und Befestigungspunkte müssten dabei neu geplant werden, weil sich Einbauwinkel und Zugänglichkeit ändern.

Vorteile

- Sehr platzsparende Integration, kaum toter Bauraum.
- Wendekreis bleibt im Normalfall unverändert, da Radstand und Lenkeinschlag nicht zwangsläufig angepasst werden müssen.
- Konstruktion wirkt insgesamt kompakter und aufgeräumter, was spätere Erweiterungen erleichtern kann.

Nachteile

- Wartbarkeit kann schlechter werden: Akkuwechsel, Schrauben nachziehen, Komponenten tauschen wird je nach Einbauposition fummeliger.
- Höherer mechanischer Anpassungsaufwand (Feilen, Ausrichten, Winkel fertigen), dadurch mehr Risiko für Passungenauigkeiten.
- Kabelmanagement wird kritischer, weil weniger Platz für saubere Radien und Zugentlastung bleibt.

4.1.2 Frankenstein Umbau

Art der Änderung

- Vorderbau wird vollständig abgeschraubt.
- Mittelteil des Chassis wird durchgesägt, um eine neue Montageposition zu schaffen.
- Der Vorderbau wird in die Mitte versetzt und dort wieder verschraubt.
- Je nach Schnittstelle müssten Verstärkungen ergänzt werden (Platten, Winkel, zusätzliche Verschraubungen), damit die Struktur unter Last nicht nachgibt.

Vorteile

- Konzeptuell einfach, weil umsetzen und festschrauben als Grundidee klar ist.
- Kann schnell einen deutlichen Effekt auf Schwerpunkt und Platzverteilung bringen, ohne viele Detailarbeiten an einzelnen Bauteilen.

Nachteile

- Wendekreis kann schlechter werden, weil sich Geometrie und Gewichtsverteilung ungünstig verändern können.
- Stabilitätsrisiko: Durchsägen schwächt das Chassis, dadurch Verwindung, Risse, lockere Verschraubungen möglich.
- Oft mehr Nacharbeit als gedacht, weil nach dem groben Umbau Verstärkungen und Ausrichtung notwendig werden.

4.1.3 Einkaufsrad Lösung

Art der Änderung

- Ein Stützrad (Einkaufsrad) wird unter dem Vorderbau montiert.
- Ziel ist, dass das Rad einen Teil der Frontlast trägt und das Fahrzeug vorne mechanisch stabilisiert.
- Befestigung über eine einfache Halterung oder Montageplatte, ohne größere Eingriffe in den Aufbau.

Vorteile

- Sehr schnell und kostengünstig umsetzbar.
- Minimal invasiv: bestehender Aufbau bleibt weitgehend erhalten, Rückbau ist einfach.
- Sofortiger Effekt auf Lastverteilung, ohne Chassis oder Hauptbaugruppen umzubauen.

Nachteile

- Optisch eher Bastellösung, wirkt weniger integriert.
- Fahrverhalten kann schlechter werden: Nachlaufen, Flattern, zusätzliche Reibung, eingeschränkte Bodenfreiheit.
- Auf unebenem Untergrund oder bei Kanten kann das Stützrad stören oder hängen bleiben.

4.1.4 Nutzwertanalyse zur Gewichtsumverteilung

Aus der Nutzwertanalyse in Tabelle 5 ergibt sich, dass die Hochkante Einlage im Mittelteil die beste Lösung zur Umverteilung des Gewichtes darstellt. Die Lösung bietet eine gute Balance aus Umsetzbarkeit, Fahrverhalten und Wartbarkeit, während sie gleichzeitig den Schwerpunkt effektiv verlagert. Obwohl der mechanische Anpassungsaufwand höher ist, überwiegen die Vorteile in Bezug auf Platznutzung und Fahrzeugperformance. Daher wird diese Lösung für die weitere Umsetzung empfohlen.

			1. Hochkante Einlage in den Mittelteil		2. „Frankenstein“ Umbau		3. Einkaufsrads Lösung	
	Wertebereich	Gewichtung	Bewertung	Nutzwert	Bewertung	Nutzwert	Bewertung	Nutzwert
Umbau aufwand	0-5	5	3	15	1	5	5	25
Zeitlicher Aufwand	0-5	7	4	28	5	35	2	14
Optik	0-5	3	4	12	5	15	2	6
Vorteile	0-5	5	4	20	4	20	3	15
Nachteile	-5-0	5	-2	-10	-4	-20	-2	-10
Nutzwertsumme				65		55		50
Rangplatz				1		2		3

Tabelle 5: Nutzwertanalyse zur Gewichtsverteilung

4.2 Gegenständen ausweichen

4.2.1 Mathematische Formeln für die Berechnung der Fahrbahn

Bei der mathematischen Modellierung von Trajektorien zur Umfahrung eines Hindernisses spielt die Wahl der zugrunde liegenden Funktion eine entscheidende Rolle. Insbesondere sind Aspekte wie Krümmungsverhalten, lokale Anpassungsfähigkeit und das asymptotische Verhalten von großer Bedeutung. Im Folgenden werden Parabeln, Exponentialfunktionen und die Gaußsche Glockenkurve im Hinblick auf ihre Eignung zur Umfahrung eines Gegenstandes verglichen.

Parabeln

Eine Parabel ist eine ganzrationale Funktion zweiten Grades der Form

$$f(x) = ax^2 + bx + c.$$

Sie besitzt genau einen Scheitelpunkt und eine konstante zweite Ableitung. Diese konstante Krümmung führt dazu, dass das Auto nicht gerade auf einen Gegenstand zu fahren kann, ihn umfahren und wieder zurück auf seine Ausgangsspur zurück fahren kann nach dem Umfahren. Eine Parabel wäre daher nur bedingt geeignet um vor einem Gegenstand umzudrehen falls der Gegenstand nicht umfahrbar ist.

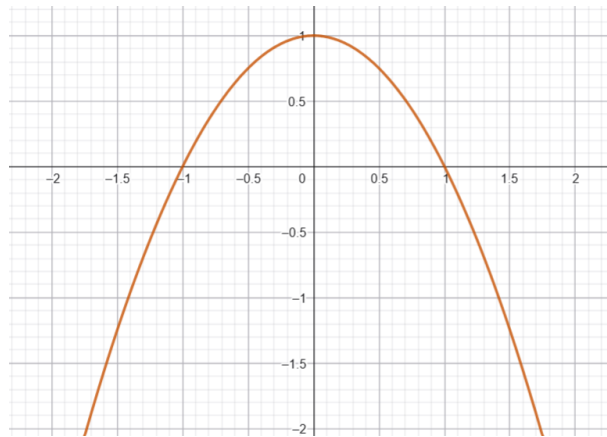


Abbildung 1: Parabel Funktion

Exponentialfunktionen

Exponentialfunktionen der Form

$$f(x) = a \cdot e^{bx}$$

zeigen ein stark asymmetrisches Verhalten. Sie besitzen keinen Scheitelpunkt und keine natürliche Symmetrieachse. Das Wachstum bzw. der Abfall erfolgt einseitig sehr stark, während auf der anderen Seite nur eine geringe Änderung stattfindet. Für die Umfahrung eines Gegenstandes ist diese Eigenschaft nachteilig, da eine ausgewogene seitliche Auslenkung benötigt wird. Exponentialfunktionen erzeugen entweder eine zu abrupte oder eine zu flache Richtungsänderung und erlauben keine kontrollierte Rückführung auf die ursprüngliche Spur. Allerdings könnten sie in speziellen Fällen, z.B. bei sehr engen Hindernissen, als Notfallmanöver eingesetzt werden, um schnell Kurven zu fahren, wenn keine andere Option besteht.

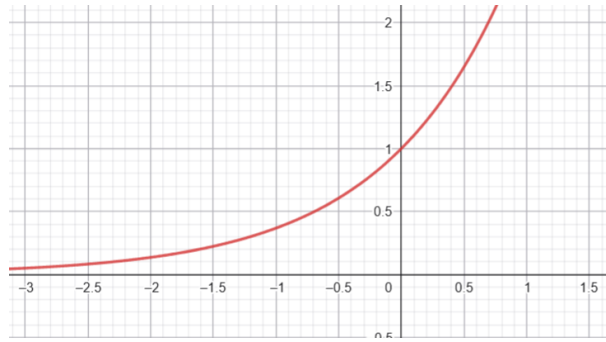


Abbildung 2: Exponential Funktion

Gaußsche Glockenkurve

Die Gaußsche Glockenkurve ist definiert durch

$$f(x) = A \cdot e^{-\frac{(x-\mu)^2}{2\sigma^2}}.$$

Sie besitzt einen klar definierten Scheitelpunkt bei $x = \mu$, ist streng symmetrisch und fällt beidseitig sanft gegen null ab. Diese Eigenschaften machen sie besonders geeignet für die Umfahrung von Hindernissen. Die Ausweichbewegung ist lokal begrenzt, kontinuierlich differenzierbar und verursacht keine abrupten Richtungsänderungen. Gleichzeitig wird der ursprüngliche Fahrweg nach dem Umfahren des Hindernisses automatisch wieder angenähert. Ein weiterer entscheidender Vorteil der Gaußschen Glocken-

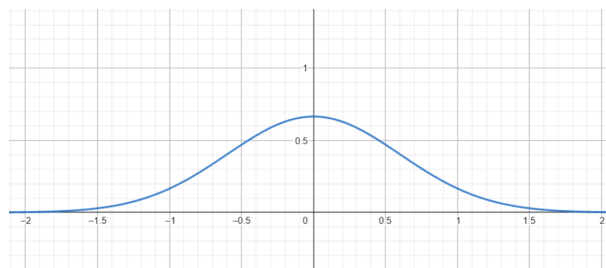


Abbildung 3: Gaussche Glocke Funktion

kurve ergibt sich durch ihre rekursive Anwendung. Wird nach der Umfahrung eines Hindernisses am Scheitelpunkt der resultierenden Kurve erneut eine Gaußfunktion angesetzt, so können auch komplexe Hindernisformen oder mehrere aufeinanderfolgende Objekte zuverlässig umfahren werden. Durch diese rekursive Überlagerung entsteht eine adaptive Fahrtenkurve, die sich flexibel an nahezu jede Umgebung anpassen kann. Unter der Voraussetzung ausreichender Parameteranpassung ist es damit theoretisch möglich, um jeden Gegenstand herumzufahren.

Fazit

Zusammenfassend lässt sich sagen, dass die Gaußsche Glockenkurve aufgrund ihrer symmetrischen Form, der lokalen Begrenzung und der sanften Krümmung die beste Wahl für die Umfahrung von Gegenständen darstellt. Parabeln können in bestimmten Situationen nützlich sein, bieten jedoch nicht die notwendige Flexibilität und Kontrolle. Exponentialfunktionen sind für diesen Anwendungsfall weniger geeignet, da sie zu einseitig und abrupt reagieren. Die Möglichkeit der rekursiven Anwendung der Gaußschen Glockenkurve ermöglicht es zudem, komplexe Hindernisformationen effektiv zu bewältigen, was sie zur optimalen Lösung für die Fahrbahnplanung bei der Umfahrung von Gegenständen macht. Damit ein optimales Fahrverhalten erreicht wird, sollten die Parameter der Gaußschen Glockenkurve sorgfältig angepasst werden, um eine harmonische und sichere Umfahrung zu gewährleisten. Dabei ist wichtig zu beachten, dass die Wahl der kurvensteigung nicht über den maximalen Lenkeinschlag hinausgehen darf, um die Fahrstabilität zu gewährleisten. Insgesamt bietet die Gaußsche Glockenkurve eine robuste und flexible Grundlage für die Entwicklung von Algorithmen zur Hindernisumfahrung in autonomen Fahrsystemen.

4.2.2 Berechnung Lenkwinkel

Die Berechnung des Lenkwinkels für die Umfahrung eines Gegenstandes basiert auf der geometrischen Beziehung zwischen der Fahrbahnkurve und der Fahrzeugdynamik. Angenommen, die Fahrbahn wird durch eine Funktion $f(x)$ beschrieben, die die seitliche Auslenkung in Abhängigkeit von der Vorwärtsbewegung x angibt, kann der Lenkwinkel δ wie folgt berechnet werden:

$$\delta(x) = \arctan \left(\frac{df}{dx} \right).$$

Hierbei ist $\frac{df}{dx}$ die erste Ableitung der Fahrbahnfunktion, die die Steigung der Kurve an einem bestimmten Punkt x angibt. Der Lenkwinkel δ wird in Radiant oder Grad angegeben und gibt an, wie stark das Fahrzeug in diesem Moment lenken muss, um der Fahrbahn zu folgen. Die Berechnung des Lenkwinkels ist entscheidend für die Steuerung des Fahrzeugs und die Gewährleistung einer sicheren Umfahrung des Gegenstandes. Es ist wichtig, dass der berechnete Lenkwinkel innerhalb der mechanischen Grenzen

des Fahrzeugs liegt, um eine Übersteuerung oder Untersteuerung zu vermeiden. Zudem sollte die Berechnung in Echtzeit erfolgen, um auf dynamische Veränderungen in der Umgebung reagieren zu können. Die Implementierung dieser Berechnung erfordert eine genaue Modellierung der Fahrzeugdynamik und eine präzise Erfassung der Fahrbahnkurve, um eine effektive und sichere Umfahrung zu gewährleisten.

4.2.3 Entwicklung der Hinderniserkennung

In einem ersten Ansatz zur Hinderniserkennung werden die vom LiDAR-Sensor bereitgestellten Messdaten direkt ausgewertet. Hierzu werden sämtliche Messpunkte innerhalb eines festgelegten Maximalabstands betrachtet. Befindet sich ein Messwert innerhalb dieses Abstandes, wird dieser unmittelbar als Hindernis interpretiert. Dieses Verfahren ermöglicht zwar grundsätzlich die Erkennung von Objekten vor dem Fahrzeug, erweist sich im praktischen Betrieb jedoch als wenig robust.

Insbesondere führen einzelne fehlerhafte Messpunkte bereits zu einer Hinderniserkennung. Solche Ausreißer entstehen beispielsweise durch Reflexionen, Messrauschen oder ungültige Distanzwerte. Das Fahrzeug beginnt dadurch teilweise ein Ausweichmanöver, obwohl sich kein tatsächliches Hindernis vor ihm befindet. Zusätzlich werden auch Objekte berücksichtigt, die sich außerhalb des relevanten Fahrbereichs befinden und somit keinen Einfluss auf die aktuelle Fahrspur haben.

Zur Verbesserung der Hinderniserkennung können mehrere Maßnahmen umgesetzt werden. Zunächst wird der ausgewertete Sichtbereich des LiDAR-Sensors auf einen Winkelbereich vor dem Fahrzeug begrenzt. Dadurch werden ausschließlich Messpunkte innerhalb des für die Fahrtrichtung relevanten Winkels berücksichtigt. Gleichzeitig wird ein maximaler Erkennungsabstand eingeführt, sodass weit entfernte Objekte die Fahrentscheidung nicht beeinflussen.

Ein weiterer wesentlicher Bestandteil ist die Clusterbildung. Hierbei wird ein Hindernis nicht mehr anhand eines einzelnen Messpunktes erkannt, sondern erst dann, wenn mehrere benachbarte Messpunkte räumlich zusammengehören. Einzelne Ausreißer werden dadurch zuverlässig unterdrückt, da sie keine ausreichende Anzahl benachbarter Messungen besitzen. Dieses Verfahren erhöht die Robustheit der Hinderniserkennung erheblich und reduziert Fehlentscheidungen.

Zusätzlich werden ungültige LiDAR-Messwerte verworfen. Hierzu zählen insbesondere

unendliche oder außerhalb des gültigen Messbereichs liegende Werte. Erst nach dieser Vorverarbeitung werden die verbleibenden Messpunkte zur Hinderniserkennung verwendet. Die Kombination aus Sichtfeldbegrenzung, Distanzfilterung und Clusterbildung bildet somit die Grundlage für eine zuverlässige Objekterkennung.

4.2.4 Implementierung der Gauß-basierten Hindernisvermeidung

Nach der Erkennung eines Hindernisses besteht die Aufgabe, dieses Hindernis zu vermeiden. Dazu soll ein geeigneter Fahrpfad berechnet und daraus ein Lenkwinkel für das Fahrzeug abgeleitet werden. Statt eines konstanten Lenkwinkels soll hierfür eine Gaußfunktion verwendet werden, um ein ruhiges Fahrverhalten zu erzeugen und nach dem Ausweichmanöver wieder auf die Ursprüngliche Spur zurück zu gelangen.

Zu Beginn eines Ausweichmanövers speichert der Controller die aktuelle Fahrzeugposition als Referenzspur (`reference_y`). Diese Referenz bildet den Ausgangspunkt für die Berechnung der Ausweichspur. Anschließend wird die Position des Hindernisses aus den vom LiDAR gelieferten Winkel- und Entfernungsinformationen in kartesische Koordinaten umgerechnet. Auf Grundlage dieser Hindernisposition wird eine Gaußfunktion erzeugt, deren Amplitude den notwendigen seitlichen Sicherheitsabstand beschreibt. Die Gaußfunktion wird anschließend auf den Referenzpunkt des Fahrzeuges addiert und ergibt so eine kontinuierliche Sollspur.

Im Gegensatz zu einer vollständigen Spurplanung wird diese Ausweichspur jedoch nicht als fester Fahrplan bis zum Ende abgefahren. Stattdessen dient sie als lokale Orientierung für die Fahrzeugregelung. Während der Fahrt wird fortlaufend ein Lookahead-Punkt auf der berechneten Spur bestimmt. Aus der seitlichen Abweichung zwischen Fahrzeugposition und diesem Zielpunkt wird mithilfe einer geometrischen Beziehung der aktuell benötigte Lenkwinkel berechnet. Dadurch reagiert das Fahrzeug kontinuierlich auf den berechneten Fahrpfad, ohne für jeden Abschnitt einen festen Lenkwinkel vorzugeben.

Die Wahl einer Gaußfunktion bietet gegenüber einer konstanten Lenkstrategie mehrere Vorteile. Durch ihren stetigen Verlauf entstehen gleichmäßige Änderungen des Lenkwinkels, wodurch abrupte Richtungswechsel vermieden werden. Gleichzeitig kann die seitliche Auslenkung über die Parameter der Gaußfunktion an unterschiedliche Hindernisgrößen angepasst werden. Die Ausweichspur ist lokal begrenzt und beeinflusst

daher nur den Bereich um das erkannte Hindernis.

Die aktuelle Implementierung stellt einen ersten funktionsfähigen Ansatz für eine Gauß-basierte Hindernisvermeidung dar. Bereits umgesetzt sind die Berechnung einer Gauß-kurve, die Bestimmung eines Lenkwinkels anhand eines Lookahead-Punktes sowie die Speicherung der ursprünglichen Fahrspur als Referenz. Eine vollständige Spurverfolgung bis zur automatischen Rückkehr auf die Ausgangsspur ist jedoch noch nicht vollständig realisiert. Sobald im aktuellen System kein Hindernis mehr erkannt wird, wird der Hindernisvermeidungscontroller zurückgesetzt. Dadurch wird die berechnete Ausweichspur nicht bis zu ihrem Ende verfolgt. Die vorhandene Referenzspur bildet jedoch bereits die Grundlage für eine spätere Erweiterung, bei der das Fahrzeug nach dem Umfahren eines Hindernisses selbstständig auf seine ursprüngliche Fahrbahn zurückkehrt.

Damit bildet die aktuelle Implementierung die Grundlage für eine kontinuierliche Hindernisvermeidung. Gegenüber einer einfachen festen Lenkstrategie ermöglicht sie bereits ein deutlich ruhigeres Fahrverhalten und schafft gleichzeitig die Voraussetzungen für zukünftige Erweiterungen, wie eine vollständige Spurverfolgung oder die Berücksichtigung mehrerer gleichzeitig erkannter Hindernisse.